# Django Caching, Cookies & Sessions

## Complete Beginner to Advanced Notes

# Part 1 — Django Caching

## 1. What Is Caching?

Caching means temporarily storing frequently accessed data so that it can be retrieved faster without repeating expensive operations.

Examples of expensive operations:

- Database queries
- External API requests
- Complex calculations
- Expensive template rendering

### Without caching

```
Request → Database / API / Calculation → Generate Result → Response
```

### With caching

```
Request → Check Cache
              |
       Cache Hit? ── Yes → Return Cached Data
              |
             No
              ↓
      Query Database / API
              ↓
        Store in Cache
              ↓
          Response
```

### Benefits

1. Faster response times
2. Reduced database load
3. Lower CPU usage
4. Better scalability
5. Improved user experience

---

## 2. Why Use Caching in Django?

Caching is useful when the same data or result is requested frequently.

Common examples:

- Product lists and product details
- Homepage content
- Popular posts
- API responses
- Dashboard statistics
- Expensive database queries
- Expensive template fragments

Caching should be applied selectively. Not every piece of data benefits from being cached.

---

## 3. Types of Django Caching

Django supports several caching approaches:

1. **Per-site cache:** Cache responses across the website.
2. **Per-view cache:** Cache the response of a specific view.
3. **Template fragment cache:** Cache only a section of a template.
4. **Low-level cache API:** Store and retrieve specific data directly.

---

## 4. Django Cache Backends

A cache backend defines where and how cached data is stored.

Common options include:

- `LocMemCache`
- `FileBasedCache`
- `DatabaseCache`
- Memcached
- Redis through a compatible backend

### 4.1 Local Memory Cache

`LocMemCache` stores cached data in local process memory.

```
CACHES = {
    "default": {
        "BACKEND": (
            "django.core.cache.backends.locmem.LocMemCache"
```

```
        ),
        "LOCATION": "unique-snowflake",
    }
}
```

Good for:

- Development
- Testing
- Small applications
- Simple local caching

Limitation: Multiple worker processes or servers may have separate caches. Cached data is not automatically shared across them.

### 4.2 File-Based Cache

File-based caching stores cached data on the filesystem.

```
CACHES = {
    "default": {
        "BACKEND": (
            "django.core.cache.backends.filebased.FileBasedCache"
        ),
        "LOCATION": "/var/tmp/django_cache",
    }
}
```

Advantages:

- Easy to configure
- No separate cache server required

Disadvantages:

- Filesystem overhead
- Generally slower than an in-memory cache
- Not automatically shared across multiple servers

Choose a suitable directory and filesystem permissions for your deployment environment.

### 4.3 Database Cache

Django can store cache entries in a database table.

```
CACHES = {
    "default": {
```

```
        "BACKEND": (
            "django.core.cache.backends.db.DatabaseCache"
        ),
        "LOCATION": "my_cache_table",
    }
}
```

Create the cache table:

```
python manage.py createcachetable
```

Database caching can be useful in certain environments, but a high volume of cache reads and writes may add load to the database.

### 4.4 Memcached

Memcached is a distributed, memory-based caching system.

Characteristics:

- Fast in-memory storage
- Suitable for shared caching
- Volatile data
- Commonly used for simple distributed caching

Django supports Memcached through its built-in Memcached backends and compatible clients.

### 4.5 Redis

Redis is an in-memory data store frequently used for caching. Depending on configuration, it can also support other workloads, such as counters, queues, and pub/sub.

A common Django integration package is:

```
pip install django-redis
```

Example configuration:

```
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.DefaultClient",
        },
```

```
    }
}
```

This example assumes Redis is available at the specified address. Production settings should reflect the actual Redis host, authentication, TLS, and deployment environment. Redis is a common production cache backend, especially when multiple Django workers need to share cached data.

---

## 5. Per-View Caching

Per-view caching stores the HTTP response produced by a particular view.

```
from django.shortcuts import render
from django.views.decorators.cache import cache_page


@cache_page(60 * 15)
def product_list(request):
    return render(
        request,
        "product_list.html",
        {},
    )
```

Here:

```
60 × 15 = 900 seconds = 15 minutes
```

Good use cases:

- Public product listings
- Public content pages
- Pages that change infrequently

**Important:** Per-view caching can vary by URL and relevant request properties. Be careful with pages containing user-specific data, authentication state, or personalized content. Do not assume a cached response is safe to share across users.

---

## 6. Template Fragment Caching

Sometimes only a small section of a page is expensive to generate.

Load the cache template tag:

```
{% load cache %}
```

Cache a fragment for 10 minutes:

```
{% cache 600 popular_products %}
    <h2>Popular Products</h2>

    <!-- Expensive template section -->
{% endcache %}
```

The fragment is cached for 600 seconds.

You can provide additional identifying arguments to separate cache entries:

```
{% cache 600 sidebar request.user.id %}
    <!-- User-specific sidebar -->
{% endcache %}
```

Use this technique only when the fragment genuinely needs to differ by user. User-specific fragments require careful privacy and invalidation decisions.

Good use cases:

- Popular product sections
- Expensive sidebars
- Repeated public content
- Expensive template components

---

## 7. Low-Level Cache API

Django provides a low-level cache API through `django.core.cache`.

```
from django.core.cache import cache
```

### 7.1 Set Cache

```
cache.set("my_key", "hello world", timeout=300)
```

The entry is cached for 300 seconds, or five minutes.

### 7.2 Get Cache

```
value = cache.get("my_key")
```

If the key exists, Django returns its cached value.

If the key is missing or expired, `None` is normally returned.

You can supply a default:

```
value = cache.get("my_key", "default value")
```

**Important:** If `None` is a legitimate cached value, use a unique sentinel object to distinguish a cache miss from a cached `None`.

```
MISSING = object()

value = cache.get("my_key", MISSING)

if value is MISSING:
    print("Cache miss")
```

### 7.3 Delete Cache

```
cache.delete("my_key")
```

### 7.4 Clear Cache

```
cache.clear()
```

Warning: `cache.clear()` may remove all entries in the configured cache backend. Avoid using it casually in production, particularly when the backend is shared.

---

## 8. Cache Keys

A cache key uniquely identifies an entry.

```
product_id = 10

cache_key = f"product_{product_id}"

cache.set(cache_key, {"name": "Phone"}, timeout=300)
```

Examples:

```
product_10
product_20
product_30
```

Use meaningful keys that describe the data being cached.

If cached results depend on a language, user, filter, or version, make sure the key or cache variation accounts for those differences.

---

## 9. Cache-Aside Pattern

Cache-Aside is a common caching strategy.

Flow:

```
1. Request arrives
2. Check cache
3. If found → return cached data
4. If missing → read database
5. Store the result in cache
6. Return the result
```

Example:

```
product = cache.get(cache_key)

if product is None:
    product = Product.objects.get(id=product_id)
    cache.set(cache_key, product, timeout=300)
```

This simple example assumes `None` cannot be a valid cached product result.

---

## 10. Cache Hit and Cache Miss

### Cache Hit

The requested value exists in the cache.

```
Request → Cache → Data Found → Return Data
```

### Cache Miss

The requested value is missing or expired.

```
Request → Cache → Not Found → Database → Store in Cache
```

A high cache hit rate can improve performance, but it should not be the only measure of cache effectiveness. Correctness, freshness, latency, and memory usage matter too.

---

## 11. Real-World Product Detail Example

```
from django.core.cache import cache
from django.shortcuts import get_object_or_404, render
from .models import Product


def product_detail(request, product_id):
    cache_key = f"product_{product_id}"

    product = cache.get(cache_key)
```

```
    if product is None:
        product = get_object_or_404(
            Product,
            id=product_id,
        )
        cache.set(
            cache_key,
            product,
            timeout=300,
        )

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

This caches the retrieved product object for five minutes.

In a larger application, you might cache a serialized dictionary of selected product fields rather than a model instance. Also consider what happens if a product does not exist or becomes unavailable.

---

## 12. Cache Invalidation

Cache invalidation means removing or refreshing cached data when the underlying source changes.

Example:

```
Database price = $100
Cache price    = $100
```

After an update:

```
Database price = $120
Cache price    = $100  ← stale
```

Delete the cached entry after a successful update:

```
cache.delete(f"product_{product_id}")
```

The next request can load the updated product from the database.

### Example update

```
from django.shortcuts import get_object_or_404, redirect
from .models import Product
from django.core.cache import cache


def update_product_price(product_id, new_price):
    product = get_object_or_404(Product, id=product_id)

    product.price = new_price
    product.save(update_fields=["price"])

    cache.delete(f"product_{product.id}")
```

This is an illustrative service-style function; adapt it to the application's view, permissions, and validation requirements.

For systems with multiple related cache keys, consider a consistent invalidation strategy. Django model signals can help in some applications, but explicit invalidation at the point where data changes is often easier to reason about.

---

## 13. Cache Timeout

A timeout controls how long a cache entry remains valid.

```
cache.set("product_10", product, timeout=300)
```

Here, the timeout is five minutes.

General guidance:

- Frequently changing data → shorter timeout
- Rarely changing data → longer timeout
- Strictly real-time data → avoid stale caching or implement a suitable refresh strategy

An expired entry should be treated as unavailable. Cache entries can also disappear earlier, so application correctness should not depend on their permanent existence.

---

## 14. Cache Strategies

### Cache-Aside

The application reads from the cache first. On a miss, it reads the source and stores the result.

### Write-Through

When data is written, the application updates the source and cache as part of its write strategy.
Write-through can keep cached values fresher but requires careful handling of partial failures and ordering.

### Background Refresh

A background job refreshes cache entries before or after expiration.
This can reduce latency spikes for expensive data, but requires appropriate scheduling and failure handling.

---

## 15. Cache Versioning

Django supports cache versions.

```
cache.set(
    "key",
    "value",
    timeout=300,
    version=2,
)

value = cache.get("key", version=2)
```

Different versions allow separate entries for the same logical key.
Versioning can help when the format or meaning of cached data changes.

---

## 16. `get_or_set()`

Django provides `cache.get_or_set()` as a convenient method.

```
value = cache.get_or_set(
    "site_name",
    "My Website",
    timeout=60,
)
```

If the key exists, the cached value is returned. Otherwise, the default is stored and returned.
A callable can be used to produce a default value:

```
value = cache.get_or_set(
    "active_product_count",
```

```
        lambda: Product.objects.filter(is_active=True).count(),
        timeout=300,
    )
```

Be aware that concurrent requests can still perform duplicate expensive work in some cache setups. `get_or_set()` alone should not be assumed to prevent every cache stampede.

---

## 17. Common Caching Mistakes

### 1. Forgetting Invalidation

Updated database data may not match the cached version.

### 2. Caching Everything

Over-caching can increase memory use, stale data, and complexity.

### 3. Choosing the Wrong Timeout

- Too short → fewer cache hits
- Too long → greater risk of stale data

### 4. Caching Sensitive Information

Avoid blindly caching passwords, authentication secrets, access tokens, payment information, or private user data.

### 5. Sharing Personalized Responses

Ensure cached pages cannot expose one user's information to another.

### 6. Assuming Cache Entries Always Exist

A cache is temporary storage. The application should handle missing entries and, where appropriate, cache failures.

---

## 18. When to Use or Avoid Caching

### Good candidates

- Frequently requested data
- Expensive database queries
- Public product lists
- Popular content
- Expensive calculations
- Repeated API results
- Expensive template fragments

### Poor candidates

- Data that must always be completely current
- Cheap calculations that do not justify cache complexity
- Highly sensitive data without a clear security design
- Data whose invalidation is harder than recomputing it

---

## 19. Production Caching Best Practices

1. Use meaningful cache keys.
2. Choose appropriate timeouts.
3. Invalidate stale data after updates.
4. Use a shared cache for multi-server deployments.
5. Monitor cache hits, misses, latency, and memory usage.
6. Do not cache everything.
7. Keep cache logic simple.
8. Consider stampede protection for expensive entries.
9. Test behavior when the cache is unavailable.
10. Avoid exposing private data through shared cached responses.

For a typical e-commerce application, a reasonable starting design might be:

```
Redis
 ├── Shared cache
 ├── Product data cache
 └── Expensive query results

Per-view cache
 └── Suitable public pages

Template fragment cache
 └── Expensive template sections

Database
 └── Authoritative product and order data
```

# Part 2 — Django Cookies

## 20. What Is a Cookie?

A cookie is a small piece of data stored by the browser and sent with relevant HTTP requests.

Cookies can be used for:

- Preferences
- Language selection
- Session identifiers
- Small client-side state

A cookie is generally limited to a few kilobytes; approximately 4 KB per cookie is a common practical guideline, although actual browser limits vary.

Do not store sensitive information directly in cookies without a deliberate security design.

---

## 21. Setting a Cookie in Django

```
from django.http import HttpResponse


def set_cookie(request):
    response = HttpResponse("Cookie set")

    response.set_cookie(
        "username",
        "Faruk",
        max_age=3600,
    )

    return response
```

The response contains a `Set-Cookie` header. The browser may store the cookie and send it on later matching requests.

`max_age=3600` specifies a lifetime of one hour.

---

## 22. Reading a Cookie

```
from django.http import HttpResponse


def get_cookie(request):
    username = request.COOKIES.get("username")

    return HttpResponse(username or "No cookie found")
```

Django provides incoming cookie values through:

```
request.COOKIES
```

**Important:** Cookie values are client-controlled input. Do not trust them for authorization, prices, roles, or other security-sensitive decisions.

---

## 23. Deleting a Cookie

```
from django.http import HttpResponse


def delete_cookie(request):
    response = HttpResponse("Cookie deleted")
    response.delete_cookie("username")

    return response
```

The deletion response should use cookie attributes consistent with the original cookie, especially its path and domain where applicable.

---

## 24. Important Cookie Security Attributes

### `HttpOnly`

Prevents normal browser JavaScript from reading the cookie.
It helps reduce cookie theft through some XSS attacks but does not prevent all XSS effects or make an application completely secure.

### `Secure`

Instructs the browser to send the cookie only over HTTPS.

### `SameSite`

Controls whether browsers send the cookie in cross-site contexts.
Common values:

- `Strict`
- `Lax`
- `None`

`SameSite=None` generally requires `Secure=True` in modern browsers.
Other important attributes include:

- `Max-Age`
- `Expires`
- `Path`
- `Domain`

---

## 25. Example of a More Secure Cookie

```
from django.http import HttpResponse


def set_cookie(request):
    response = HttpResponse("Cookie set")

    response.set_cookie(
        "username",
        "Faruk",
        max_age=3600,
        httponly=True,
        secure=True,
        samesite="Lax",
    )

    return response
```

This example is appropriate for HTTPS deployments. During local HTTP development, `secure=True` may prevent the browser from sending the cookie.

For authentication-related cookies, use Django's built-in authentication and session facilities rather than inventing a custom authentication cookie.

# Part 3 — Django Sessions

## 26. What Is a Session?

A session allows Django to maintain user-specific state across multiple requests.

With a typical server-side session backend:

```
Browser
   |
   | sessionid cookie
   ↓
Django
   |
   | Look up session
   ↓
Session Store
   ├── User preferences
   ├── Temporary state
   └── Other session data
```

The browser typically stores a session identifier. The actual session data is stored by the configured session backend.

Django also supports a signed-cookie session backend, which stores session data in a signed cookie rather than in a server-side session store. Signed data is not automatically encrypted.

---

## 27. How Django Sessions Work

A typical session flow:

1. The user sends a request.
2. Django's session middleware makes session data available.
3. The application reads or updates the session.
4. Django saves changes when necessary.
5. The browser receives or retains the session cookie.
6. Later requests send the cookie.
7. Django uses the session identifier to retrieve session data.

The precise behavior depends on the session backend and cookie settings.

---

## 28. Where Does Django Store Sessions?

With the standard database-backed session configuration, Django commonly stores session records in:

```
django_session
```

Other supported session engines include:

- Database-backed sessions
- Cache-backed sessions
- Cached database sessions
- File-based sessions
- Signed-cookie sessions

Session middleware and an appropriate session engine must be configured for `request.session` to work.

For the database-backed session engine, apply migrations:

```
python manage.py migrate
```

---

## 29. Setting Session Data

```
from django.http import HttpResponse


def set_session(request):
    request.session["username"] = "Faruk"
    request.session["role"] = "user"

    return HttpResponse("Session set")
```

Session data behaves like a dictionary.
Do not use a session value such as `"role": "admin"` as your application's sole authorization mechanism. Use Django's authentication, permissions, and trusted server-side user records.

---

## 30. Getting Session Data

```
from django.http import HttpResponse


def get_session(request):
    username = request.session.get("username")

    return HttpResponse(username or "No username")
```

Use `.get()` when the key might not exist.

---

## 31. Deleting Session Data

Delete one key:

```
del request.session["username"]
```

However, this raises `KeyError` if the key is missing.
A safer option is:

```
request.session.pop("username", None)
```

---

## 32. Clearing the Entire Session

```
request.session.flush()
```

This removes the current session data and deletes the current session record where applicable, while clearing the current session key.

Use this when the current session should be fully discarded, such as during a security-sensitive session reset.

For normal login and logout workflows, prefer Django's built-in authentication functions.

---

## 33. Session Expiry

Set the session lifetime in seconds:

```
request.session.set_expiry(300)
```

This sets the session to expire after five minutes of inactivity under Django's session-expiry behavior.

To use a browser-session cookie:

```
request.session.set_expiry(0)
```

To use the configured default expiry:

```
request.session.set_expiry(None)
```

The browser-session cookie behavior depends on browser behavior and cookie configuration. Closing a browser does not guarantee that every browser will immediately discard every session cookie.

---

## 34. Django Authentication and Sessions

Django authentication commonly uses sessions to maintain login state.

A simplified example:

```
from django.contrib.auth import authenticate, login


def login_example(request):
    username = request.POST.get("username")
    password = request.POST.get("password")

    user = authenticate(
        request,
        username=username,
        password=password,
    )

    if user is not None:
```

```
            login(request, user)
            # The user is now authenticated.

        ...
```

This is an illustrative snippet. A real login view should validate input, handle invalid credentials, return an appropriate response, and use Django's CSRF protection for POST forms.

After authentication, the current user is generally available through:

```
request.user
```

This requires Django's authentication middleware and related settings to be configured.

---

### 35. Logging Out

Use Django's built-in logout function:

```
from django.contrib.auth import logout
from django.shortcuts import redirect


def logout_view(request):
    logout(request)
    return redirect("login")
```

Replace `"login"` with the actual URL name in your application.

Logout should normally be triggered by a POST request, not a GET request, to avoid unintended state changes through links or crawlers. A production implementation should use a CSRF-protected form.

## Part 4 — Cookies vs Sessions

### 36. Cookie vs Session

| Feature | Cookie | Session |
| --- | --- | --- |
| Typical storage | Browser | Configured session backend |

| | | |
|---|---|---|
| Typical client-side value | Cookie data or identifier | Session ID cookie for server-side backends |
| Size | Limited | Depends on backend and application limits |
| Server-side state | Not inherently | Usually yes with server-side backends |
| Common use | Preferences, identifiers | Login state, temporary user state |
| Django API | `request.COOKIES` | `request.session` |

Do not assume cookies are always insecure or sessions are automatically secure. Both require appropriate security configuration.

---

## 37. Django's `sessionid` Cookie

With the usual server-side session configuration, Django commonly uses a cookie named:

```
sessionid
```

Conceptually:

```
Browser:
    sessionid=...

Django session store:
    session key → associated session data
```

The actual session data normally remains in the configured server-side store when using database-backed or cache-backed sessions.

The session identifier is credential-like information. Do not share it publicly or place it in logs unnecessarily.

---

## 38. Inspecting Session Cookies in Chrome

1. Open your Django site in Chrome.
2. Open DevTools using `F12` or right-click and select **Inspect**.
3. Select the **Application** tab.
4. Under **Storage**, open **Cookies**.
5. Select the relevant website origin, such as `http://127.0.0.1:8000` during local development.
6. Inspect the cookie named `sessionid`, if present.

You may see columns such as:

- Name
- Value
- Domain
- Path
- Expires / Max-Age
- HttpOnly
- Secure
- SameSite

Do not publish session cookie values in screenshots, bug reports, or source-control repositories.

---

# Part 5 — Caching vs Sessions

## 39. Cache vs Session

### Cache

Primarily used for:

- Improving performance
- Reducing repeated database queries
- Reducing computation
- Reducing response time

### Session

Primarily used for:

- Maintaining user-specific state
- Login state
- Temporary user preferences
- Shopping cart state in simple applications

A simple rule:

```
Need faster repeated reads? → Cache

Need state associated with a user? → Session

Need durable business data? → Database
```

A shopping cart that must persist across devices, support multiple concurrent sessions, or reliably survive session expiry is often better stored in the database, possibly with a session used to identify an anonymous cart.

---

# Part 6 — Real-World E-Commerce Architecture

## 40. Example Architecture

```
Browser
   |
   | Cookies / Session ID
   ↓
Django Application
   ├── Authentication
   ├── Session Management
   ├── Cache Layer
   └── Business Logic
          |
          ├── Redis Cache
```

```
            |     ├── Product data
            |     └── Expensive query results
            |
            └── Database
                  ├── Users
                  ├── Products
                  ├── Orders
                  └── Shopping carts
```

Example choices:

| **Feature** | **Common approach** |
| --- | --- |
| Login state | Django authentication and sessions |
| Product list | Cache where useful |
| Product detail | Low-level cache or suitable view caching |
| Shopping cart | Session or database, depending on requirements |

| | |
|---|---|
| Expensive query | Low-level cache |
| Expensive template section | Template fragment cache |
| Orders and payments | Database as the authoritative source |

---

## 41. Product Detail with Cache

```
from django.core.cache import cache
from django.shortcuts import get_object_or_404, render
from .models import Product


def product_detail(request, product_id):
    cache_key = f"product_{product_id}"

    product = cache.get(cache_key)

    if product is None:
        product = get_object_or_404(
            Product,
            id=product_id,
        )

        cache.set(
            cache_key,
            product,
            timeout=300,
        )

    return render(
        request,
        "product.html",
```

```
        {"product": product},
    )
```

This example caches an existing product for five minutes. For a production application, decide how to handle missing products, cache failures, related objects, and stale values.

---

## 42. Invalidating Product Cache

After successfully updating a product:

```
product.price = new_price
product.save(update_fields=["price"])

cache.delete(f"product_{product.id}")
```

The next request will encounter a cache miss and reload the current product.
If the product appears in other cached results, such as a homepage or category listing, those related cache entries may also need invalidation.

---

## 43. Cache Failure

A robust application should consider what happens when the cache becomes unavailable.

Possible approaches:

- Fall back to the database for suitable read operations.
- Log cache failures.
- Monitor backend health.
- Use sensible timeouts and retry behavior.

Whether to fall back depends on the backend, workload, and availability requirements.
Do not assume every cache error should be silently ignored.

---

## 44. Cache Stampede

A cache stampede occurs when many requests try to rebuild the same expensive cache entry after it expires or disappears.

```
Popular cache entry expires
        ↓
Many requests get a cache miss
        ↓
Many database queries run simultaneously
```

```
          ↓
Database load spikes
```

Possible solutions:

- Locks or single-flight request coordination
- Staggered expirations
- Background refresh
- Serving stale data where appropriate
- Prewarming important cache entries

A cache timeout alone does not prevent a stampede.

---

## 45. Security Best Practices

### Caching

- Avoid caching private data in a shared cache unless keys and access controls isolate it correctly.
- Avoid caching secrets without a clear reason.
- Use explicit cache keys and appropriate expiry.
- Ensure personalized responses cannot leak between users.
- Invalidate entries when underlying data changes.

### Cookies

- Use `Secure=True` over HTTPS.
- Use `HttpOnly=True` for cookies that do not need JavaScript access.
- Choose an appropriate `SameSite` setting.
- Do not trust client-supplied cookie values.
- Avoid storing sensitive information directly in cookies.

### Sessions

- Use Django's built-in authentication system.
- Configure secure session cookie settings in production.
- Do not store passwords or unnecessary secrets in sessions.
- Use CSRF protection for state-changing forms.
- Use Django's login and logout functions where appropriate.

Useful production settings for an HTTPS site include:

```
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
```

```
SESSION_COOKIE_SAMESITE = "Lax"

CSRF_COOKIE_SECURE = True
```

These are common starting points, not a complete security configuration. Review the Django security documentation and your deployment requirements before changing cookie settings.

---

# Part 7 — Important Terms

| Term | Meaning |
|---|---|
| Cache | Temporary storage for faster access |
| Cache hit | Requested data is found in cache |
| Cache miss | Requested data is not found in cache |
| Cache key | Identifier for a cache entry |
| Timeout | Cache entry lifetime |
| Invalidation | Removing or |

| | |
|---|---|
| | refreshing stale cached data |
| Cache backend | System used to store cached data |
| Redis | In-memory data store often used for caching |
| Cookie | Small browser-stored data sent with matching requests |
| Session | Mechanism for maintaining state across requests |
| Session ID | Identifier used by server- |

| | |
|---|---|
| | side session backends |
| `HttpOnly` | Prevents ordinary JavaScript access to a cookie |
| `Secure` | Restricts cookie transmission to HTTPS |
| Cache stampede | Many requests rebuild the same expired cache entry |

---

# Part 8 — Interview Questions and Answers

### Q1. What is caching?

Caching temporarily stores frequently accessed data so future requests can retrieve it faster.

### Q2. Why use caching in Django?

To reduce repeated database queries, expensive computation, response time, and server load.

### Q3. What are Django's main caching approaches?

- Per-site cache

- Per-view cache
- Template fragment cache
- Low-level cache API

### Q4. What is a cache backend?

The storage system used to store cache entries, such as local memory, files, a database, Memcached, or Redis through a compatible backend.

### Q5. What is a cache hit?

The requested value is found in the cache.

### Q6. What is a cache miss?

The requested value is unavailable in the cache, so the application may need to rebuild or retrieve it from another source.

### Q7. What is cache invalidation?

Removing or refreshing cached data after the underlying data changes.

### Q8. What is Cache-Aside?

A strategy where the application checks the cache first, reads the source on a miss, and stores the result in the cache.

### Q9. What is a cookie?

A small piece of data stored by the browser and sent with matching requests.

### Q10. What is a Django session?

A mechanism for maintaining state across multiple requests, commonly using a session identifier cookie and server-side storage.

### Q11. Where is Django session data stored?

With the default database-backed session configuration, session records are stored in the `django_session` table. Other session engines can be configured.

### Q12. What is `sessionid`?

A common cookie name used to identify a Django session with the standard server-side session setup.

### Q13. What is the difference between cookies and sessions?

Cookies are browser-stored data. Sessions maintain request-to-request state and commonly store the associated data server-side.

### Q14. What does `HttpOnly` do?

It prevents ordinary browser JavaScript from reading a cookie. It does not eliminate every XSS risk.

### Q15. What does `Secure` do?

It instructs the browser to send the cookie only over HTTPS.

### Q16. What is `cache.get_or_set()`?

It returns an existing cached value or stores and returns a default when the key is missing. Concurrent expensive computations may still require additional coordination.

### Q17. What is a cache stampede?

A sudden spike in repeated expensive work when many requests rebuild the same expired cache entry.

### Q18. Should every product query be cached?

No. Cache data when the performance benefit justifies the extra complexity of expiry, invalidation, memory usage, and freshness management.

---

# Part 9 — Quick Revision

## Caching

```
from django.core.cache import cache

cache.set("key", "value", timeout=300)
value = cache.get("key")
cache.delete("key")
```

```
Per-view cache      → Cache a view response
Fragment cache      → Cache a template section
Low-level cache API → Cache specific data
```

## Cookies

```
response.set_cookie(
    "theme",
    "dark",
    max_age=3600,
    httponly=True,
    secure=True,
    samesite="Lax",
)
```

```
theme = request.COOKIES.get("theme")
response.delete_cookie("theme")
```

### Sessions

```
request.session["username"] = "Faruk"

username = request.session.get("username")

request.session.pop("username", None)

request.session.set_expiry(300)
```

Use Django's built-in authentication functions for login and logout.

---

# Final Mental Model

```
USER
  |
  v
BROWSER
  ├── Cookies
  └── Session ID cookie
          |
          v
DJANGO APPLICATION
  ├── Session middleware → Session state
  ├── Cache API           → Faster repeated reads
  └── Views and business logic
          |
          ├── Cache backend (e.g., Redis)
          └── Database (authoritative business data)
```

### Golden Rules

1. **Need better performance?** Consider caching.
2. **Need user-specific state across requests?** Use sessions where appropriate.
3. **Need small browser-side data?** Use cookies carefully.
4. **Need durable business records?** Use a database.
5. **Need current data?** Design a clear cache invalidation or refresh strategy.
6. **Need secure authentication?** Prefer Django's built-in authentication and session mechanisms.

7. **Need production scalability?** Use a suitable shared cache backend and monitor its behavior.

Remember: caching improves performance, cookies store browser-side data, and sessions maintain state across requests. They solve different problems and can work together in the same Django application.